

Cardinality Estimation 实验报告

1. 任务理解

Cardinality Estimation 的目标是：给定一条 SQL 风格查询的结构化表示，包括 `Tables`、`Join Conditions` 和 `Predicates`，预测该查询最终结果集的基数（即返回行数）。这类问题的核心难点在于，不同表之间存在相关性，不同谓词之间也并非独立，因此仅靠简单的独立性假设通常会产生较大误差。

本实验采用的主要评测指标是 Q-error。对于真实基数 y 和预测基数 $\hat{y}$ ，定义为：

$$Q\text{-error} = \max\left(\frac{\hat{y}}{y}, \frac{y}{\hat{y}}\right)$$

Q-error 越接近 1 越好。实验中主要报告 `mean Q-error`、`median Q-error`、`p90`、`p95` 和 `max Q-error`。其中：

- `mean Q-error` 反映总体平均误差；
- `median Q-error` 反映典型样本误差；
- `p95 Q-error` 反映尾部困难样本的稳定性；
- `max Q-error` 用于观察最极端失败案例。

2. 数据处理

2.1 输入字段解析

每条查询样本包含三个核心字段。

1. `Tables`

格式如 `title t, movie_companies mc`。解析时按逗号切分，再提取：

- 原始表名，如 `title`、`movie_companies`
- 表别名，如 `t`、`mc`

后续实现中主要使用表别名构造组合特征，因为别名与统计文件中的列名前缀直接对应。

2. `Join Conditions`

格式如 `t.id=mc.movie_id,t.id=mk.movie_id`。解析时按逗号切分为 join 条件列表，用于构造：

- join 数量特征
- 排序后的 join 签名特征

当前实现没有对 join 条件做更细粒度的图结构建模，而是保留为计数和签名。

3. `Predicates`

格式为三元组串联，例如

`ci.person_id,=,1423339,ci.role_id,=,1`。

解析时每 3 个字段组成一个谓词三元组（列名，操作符，常数值），操作符只考虑 `<`、`=`、`>` 三类。

2.2 列统计信息的使用

`column_min_max_vals.csv` 为每一列提供:

- `min`
- `max`
- `cardinality`
- `num_unique_values`

这些统计信息在特征构造中有四个主要作用。

1. 数值归一化

对谓词值 v 计算:

$$\text{norm} = \frac{v - \min}{\max(\max - \min, 1)}$$

2. 估计单谓词选择率

- 对等值谓词 `=`, 用 `1 / num_unique_values` 近似;
- 对范围谓词 `<` 或 `>`, **假设列值近似均匀分布**, 用区间比例估计; 同时**设置下界** `1 / cardinality`, 避免选择率退化为 0。

3. 构造表规模特征

对每个出现的表别名, 读取其主键列 `<alias>.id` 的 `cardinality`, 构造多表基数乘积的对数近似。

4. 构造独立性估计特征

将“表基数乘积”与“谓词选择率乘积”组合, 得到一个基于独立性假设的粗略对数基数估计, 作为模型输入特征, 而不是直接作为最终预测。

3. 特征设计

最终训练特征维度为 261 维。

3.1 核心特征

特征可分为六类。

1. 查询结构特征

- `n_tables`, `n_joins`, `n_predicates`

2. 表组合特征

- 每个候选表别名的出现标记, 如 `has_table_t`
- `table_combo`: 按别名拼接后的表组合签名

3. Join 签名特征

- `join_combo`: 将 join 条件排序后拼接成稳定字符串

4. 谓词列级特征

对每个预定义列, 如 `t.production_year`、`mc.company_id`、`ci.person_id` 等, 构造:

- 是否出现, 出现次数
- 各操作符计数
- 最小/最大/平均归一化值

- 最小选择率，对数选择率和

5. 全局选择率与独立估计特征

- `sum_log_selectivity`, `product_selectivity`, `min_selectivity`, `max_selectivity`, `mean_selectivity`
- `log_product_table_card`, `mean_log_table_card`, `independent_log_card_estimate`

6. 等值谓词目标统计特征

在训练集内部，先统计“某个等值谓词值历史上对应的目标基数分布”，再把这些统计作为特征回灌给模型，包括：

- 等值谓词出现次数
- 对应目标 `log(cardinality+1)` 的均值、中位数、分位数
- 在特定表组合上下文中的中位数统计
- 等值值频率的排序位置

这部分特征是当前实现中最关键的经验增强之一，因为很多查询的难点来自“某个具体 ID 值对应的规模模式”。

3.2 归一化

数值谓词的常数使用基于列统计的 min-max 归一化。这样做有两个目的：

- 消除不同列量纲差异；
- 让模型更容易比较不同列中“值所处的位置”。

对选择率相关特征使用对数变换，如 `sum_log_selectivity`，以减弱连乘后数值过小的问题。

目标值使用 `log1p(Cardinality)` 作为回归目标，缓解基数分布的长尾性。

3.3 one-hot 处理

本实现**没有使用 one-hot 编码**。

原因是 `table_combo`、`join_combo`、`predicate_signature`、`full_signature` 这类签名特征基数较高，如果直接 one-hot，会迅速扩大维度，并导致特征非常稀疏。

因此实现中采用的是：

- 先在训练集上为每个类别值建立稳定整数映射；
- 训练集中的已见类别映射到正整数；
- 目标集中的未见类别统一映射为 0。

由于后续模型是树模型 XGBoost，这种整数编码可以被直接利用，不需要额外兼容 one-hot 稀疏表示。

3.4 缺失值处理

缺失值处理策略比较直接：

- 类别特征缺失时填充为 `"NONE"`；
- 数值特征在最终对齐后统一 `fillna(0.0)`；
- 目标集中不存在于训练集类别映射中的新类别编码为 0。

这种处理方式保证了训练集与验证/测试集列完全对齐，不会因为字段缺失导致特征维度不一致。

3.5 异常值处理

实现中没有对原始谓词值做额外截断，而是通过派生特征降低异常值影响：

- 选择率估计被限制在 `[1/cardinality, 1]` 范围内；
- 预测输出在从 log 空间还原时，log 值被截断到 `[0, 25]`，避免指数还原时溢出；
- 最终预测基数至少为 1。

因此，这套方案没有选择“删掉异常值”，而是“限制其在派生特征和输出空间中的极端传播”。

4. 模型与训练

4.1 总体思路

整体不是单一模型，而是一个多层候选流水线：

1. 主回归模型 `main_model`
2. 低基数分类器 `low10_classifier`
3. 低基数分类器 `low100_classifier`
4. 低基数专家回归器 `low_expert`
5. 残差修正模型 `residual`
6. 固定比例融合模型 `blend_low_020`

最终提交使用固定候选 `blend_low_020`。

4.2 训练/验证划分

在 `train.csv`（总数据量为 50000）上先做一次内部验证：

- `test_size=0.2`
- `random_state=20260519`
- 按 `Cardinality` 的分位桶分层抽样

也就是：

- 80% 作为 `dev`，大小为40000
- 20% 作为 `val`，大小为10000

内部验证的作用有两个：

- 评估不同候选模型的效果；
 - 此处尝试的过程中**对树的最大深度、损失函数、学习率、正则化等进行了超参调试**，通过比较最终的mean Q-error，我选择使用下面的`main_model`作为主回归模型
- 决定主模型最佳迭代轮数，并观察哪种候选更适合作为最终方案。

4.3 主回归模型 `main_model`

主模型使用 XGBoost 回归，目标为 `log1p(Cardinality)`。

参数如下：

```
- objective = reg:squarederror
- eval_metric = rmse
- tree_method = hist
- eta = 0.03
- max_depth = 7
- min_child_weight = 1.5
- subsample = 0.92
- colsample_bytree = 0.92
- lambda = 1.2
- alpha = 0.02
- num_boost_round = 3200
- early_stopping_rounds = 160
```

训练方式：

- 在内部验证阶段，用 `dev` 训练、`val` 早停；
- 记录最佳轮数 `best_iteration=1487`；
- 全量训练阶段，不再重新搜索轮数，而是采用 `1488` 轮左右的固定轮数在全量训练集上重新训练。

这样做的原因是：先用内部验证确定合理训练长度，再把全部训练数据用于最终提交模型。

4.4 低基数分类器

为了专门处理小基数查询，额外训练两个二分类器：

1. `low10_classifier`

- 标签： `Cardinality <= 10`

2. `low100_classifier`

- 标签： `Cardinality <= 100`

二者均使用 XGBoost 二分类，主要参数为：

```
- objective = binary:logistic
- eval_metric = logloss
- eta = 0.035
- max_depth = 5
- min_child_weight = 2.0
- subsample = 0.9
- colsample_bytree = 0.9
- lambda = 2.0
- alpha = 0.05
```

类别不平衡通过 `scale_pos_weight = negatives / positives` 自动调整。

训练方式：

- 输入特征与主回归模型相同；
- 输出的是目标集属于低基数区间的概率，而不是直接输出基数值；
- 在最终阶段分别训练 600 轮。

4.5 低基数专家回归器 low_expert

该模型只在低基数样本子集上训练，使用 `Cardinality <= 100` 的训练样本（如果样本数不足 50，则放宽为 `Cardinality <= 1000`，但本实验中样本足够）

参数为：

```
- eta = 0.04
- max_depth = 4
- min_child_weight = 1.0
- subsample = 0.95
- colsample_bytree = 0.95`
- lambda = 1.0`
- alpha = 0.02
```

训练目标同样是 `log1p(Cardinality)`。

训练完成后，不是直接把 `low_expert` 作为最终输出，而是根据两个低基数分类器的概率进行软融合（此处超参数也是经过了本地多轮调参选定的结果较优异的结果）：

$$w_{low} = 0.65 \cdot P(y \leq 100) + 0.35 \cdot P(y \leq 10)$$

$$\hat{y}_{low_expert} = w_{low} \hat{y}_{expert} + (1 - w_{low}) \hat{y}_{main}$$

这一层的意义是：当前面的两个分类器判断“这条查询很可能属于低基数区域”时，更多依赖低基数专家的结果；否则退回主模型。

4.6 残差修正模型 residual

残差模型的目标不是直接预测基数，而是修正主模型误差。

训练流程分两步。

第一步：为训练集生成主模型的 OOF 预测

- 使用 3 折 `KFold`
- 每一折只用该折之外的数据训练主模型
- 对当前折生成 out-of-fold 基础预测

这样做是为了避免“用同一个样本训练后再预测自己”，从而让残差标签更接近真实泛化误差。

第二步：训练残差回归器

定义残差标签：

$$r = \log(1 + y) - \hat{y}_{main, OOF}$$

并在原始特征上额外加入：

- `main_log_prediction`
- `main_cardinality_prediction`

再训练一个 XGBoost 回归器去拟合残差。预测时：

$$\hat{y}_{residual} = \hat{y}_{main} + \hat{r}$$

这层的作用是让模型显式学习“主模型在什么区域容易系统性高估或低估”。

4.7 最终融合模型 `blend_low_020`

最终提交的模型是低基数专家回归模型和残差修正模型结果的混合，名为 `blend_low_020`。：

$$\hat{y}_{final} = 0.2 \cdot \hat{y}_{low_expert} + 0.8 \cdot \hat{y}_{residual}$$

其中使用的超参同样通过本地调参以及多次提交找到最佳分数对应权重来确定。

选择这个设计的理由是：

- `residual` 在整体上Q-error表现更优；
- `low_expert` 对极低基数样本有补充价值，能够比较好的减小极低基数样本的预测Q-error；
- 少量引入低基数专家有助于降低平均误差，同时不过度牺牲整体稳定性。

4.8 调参策略

当前代码实现为前期网格搜索后的结果，大部分模型的超参都是通过**对照本地验证结果和提交成绩反复调整**得到的：

- 首先通过比较多个主模型训练结果的mean Q-error确定一套**主模型参数**；
- 再通过内部验证观察主模型、低基数专家、残差模型和融合模型的相对收益，从而确定当前的**模型架构**；
- 最后再通过更加细粒度的网格搜索确定**低基数专家、残差模型和融合模型的超参**。

而对于训练轮数，则是通过**内部训练经验迁移**以及**保守设定**

- 主模型轮数通过 early stopping 确定；
- 全量训练阶段复用验证阶段得到的最佳轮数；
- 辅助模型轮数手动设置为较保守的固定值，防止在全量训练上过拟合。

5. 实验结果

5.1 本地验证结果

根据 `validation_metrics.json` 和 `candidate_metrics.csv`，内部验证结果如下：

候选模型	Mean Q-error	Median	P90	P95	Max
<code>main_model</code>	3.7290	1.1816	3.5190	7.4590	5447
<code>low_expert</code>	4.1217	1.1834	4.0348	9.2381	5052
<code>residual</code>	3.3934	1.1745	3.5000	7.2325	3897
<code>blend_low_020</code>	3.4036	1.1763	3.5872	7.5018	4105

可以看到：

- 单独主模型已经能达到较强基线；
- 低基数专家单独使用时整体反而更差，说明它只适合作为结果是低基数时的局部补充；
- 残差模型明显优于主模型，说明主模型存在可学习的系统误差；

- `blend_low_020` 在 `mean Q-error` 上与残差模型非常接近，在提交kaggle之后的分数表现最佳，因此被作为最终提交候选。这一定程度上说明了低基数专家模型在处理测试集中的低基数查询时起到了一定的作用。

5.2 Kaggle 分数

最终提交为 `blend_low_020` 产生的预测结果，Kaggle 分数为：

- `2.5920`

这说明本地验证趋势与线上表现基本一致：残差修正加少量低基数专家融合，优于单独主模型，也略优于单独残差修正模型。

5.3 主要对比实验结论

本实验的主要对比可以概括为三点：

- 从 `main_model` 到 `residual`，说明“先预测再学残差”是有效的；
- `low_expert` 单独效果不好，但与主模型做概率加权后能提供补充；
- 最终 `0.2 / 0.8` 固定融合相比单独 `residual` 只有小幅提升，说明收益主要来自残差修正，低基数专家属于边际改进。

6. 误差分析

在进行内部训练后，我会记录 `validation_errors.csv` 【内部验证集上按 Q-error 排序的逐条误差明细】。从中抽取若干高 Q-error 样本进行误差分析，发现失败场景集中在以下几个方面。

6.1 谓词为范围条件时被严重高估

典型样本：

- `Id=22977`
- 真实基数 `1`
- 预测 `4105`
- Q-error `4105`

查询特点：

- 多表 join
- 谓词全是范围条件，如 `t.production_year < 1915`、`mc.company_id < 1310`

原因分析：

- 范围谓词选择率是按列独立、近似均匀分布估计的；
- 真实数据中这些条件联合作用后可能几乎没有结果；
- 但模型更容易从历史样本学到“这一类结构通常不至于为 1”，因此出现大幅高估。

6.2 具体 ID 等值查询时被严重低估

典型样本：

- `Id=39016`
- 条件：`ci.person_id = 1423339` 且 `ci.role_id = 1`

- 真实基数 22671
- 预测 12
- Q-error 1889.25

原因分析：

- 这类查询高度依赖具体 `person_id` 的语义；
- 如果该值在训练中很少出现或统计不稳定，模型就只能退回到“普通等值过滤”模式；
- 结果会把某些头部人物对应的大规模结果误判成普通长尾 ID，导致结果被低估。

6.3 单表稀有值等值查询时被严重高估

典型样本：

- `Id=29432`
- 条件: `mk.keyword_id = 636`
- 真实基数 1
- 预测 621

原因分析：

- 尽管实现引入了等值统计特征，但它只在训练集中出现次数达到阈值时才会保留；
- 对未见或低频值，模型很难分辨“极稀有值”和“普通值”；
- 因此会回归到平均模式，导致对真实值为 1 的样本明显高估。

6.4 强相关 join 场景被严重低估

典型样本：

- `Id=13641`
- 真实基数 61413
- 预测 535
- Q-error 114.79

查询特点：

- 涉及 `title`、`movie_companies`、`cast_info`
- 包含多个等值过滤

原因分析：

- 当前特征虽然编码了表组合、join 签名和谓词签名，但没有显式建模 join 后重复度和多列相关性【模型以独立均匀分布为前提】；
- 当多个条件共同指向一个密集区域时，模型仍可能把它当作“多条件收缩后的稀疏结果”，从而大幅低估。

6.5 当前方法的主要瓶颈

综合高误差样本，瓶颈主要有：

- 范围谓词选择率估计过于粗糙，隐含了均匀分布假设；
- 对未见或低频等值的泛化仍然不足；
- join 相关性只通过签名隐式表达，没有更细的结构统计；

7. 复现说明

复现实验所需文件：

- `train.csv`
- `test.csv`
- `sample_submission.csv`
- `column_min_max_vals.csv`

复现依赖的库及对应版本

- `numpy==2.4.4`，用于数值计算、对数变换、Q-error 统计
- `pandas==3.0.2`，用于读取 CSV、构造特征表、导出结果
- `scikit-learn==1.8.0`，用于`train_test_split`、`KFold`
- `xgboost==3.2.0`，用于主回归模型、低基数分类器、低基数专家、残差模型的构建
- 当前代码还依赖若干 Python 标准库，包括 `argparse`、`json`、`math`、`pathlib`、`time`、`collections`、`dataclasses`。

执行脚本：

- `solve_cardinality.py`

核心流程：

1. 读取训练集、测试集和列统计；
2. 在训练集上做 8:2 内部验证；
3. 构造 261 维特征；
4. 训练主模型、低基数分类器、低基数专家、残差模型；
5. 用全量训练集重新训练，并输出最终 `submission.csv`；
6. 同时保存 `validation_metrics.json`、`candidate_metrics.csv` 和 `validation_errors.csv`。

8. 总结

本实验采用了“统计特征 + XGBoost 多层建模”的方案。主模型负责学习整体基数分布，低基数分类器和低基数专家负责补充小基数区域，残差模型负责纠正主模型系统误差，最终再进行固定比例融合。实验结果表明，残差修正是最主要的收益来源，而低基数专家带来小幅增益。最终模型在本地验证集上取得 `mean Q-error = 3.3171`，并在 Kaggle 上取得 `2.6518` 的成绩。

9. AI 工具使用说明

本实验在代码实现、检查和文档整理阶段使用了 AI 工具辅助，但最终设计选择、结果判断和提交内容均经过人工核查。

1. 使用方式

AI 工具主要用于以下环节：

- 对设想的模型进行代码实现与结果验证
- 在实现基础模型后提供进一步的优化方向
- 根据代码生成实验报告初稿结构

2. 关键流程

实际使用流程如下：

1. 构建模型

1. 先由人工确认作业要求与题目内容，通过网上查找论文，找到可能适用于cardinality预测的xgboost模型和nn模型。
2. AI进行代码实现，通过训练和结果比较确定使用xgboost作为主模型
3. 在与AI的交互中进一步探索可能的优化方向，找到通过补充低基数专家模型和残差模型优化预测结果的方案，并由AI实现
4. 在AI的帮助下对模型进行超参调试
5. 在以上过程中，人工阅读代码理解语义，删除一些不必要的调错/结果输出文件，对代码进行简单注释。

2. 完成报告

1. 由 AI 辅助阅读文件，总结训练与推理流程。
2. 基于代码与结果，AI生成实验报告文本初稿。
3. 人工完善实验报告，精化表述，核对正确性

3. 人工检查与思考过程

在整个过程中，我主要完成了以下内容

- 确定适用于该场景的基础模型，基于训练结果决定模型的高级架构与优化方向
- 对训练结果和关键结论进行回溯核查，对AI生成代码阅读、充分理解并注释，确保正确性
- 对实验报告的表述进行精确化、细化，核对表述的严谨性与正确性

4. 使用边界

AI 工具没有替代以下工作：

- 没有替代人工决定最终模型架构
- 没有替代人工核对指标与输出结果（提交文件）